ALA - 2 : Python Deep Dive: Interview Intensive

Topic: Tuple Packing and Unpacking in Python

Interview Question And Answers

1. What is tuple packing in Python?

Answer: Tuple packing means storing multiple values into a single tuple.

Example:
t = 10, 20, 30

2. Explain tuple unpacking with syntax.

Answer: Tuple unpacking assigns tuple values to variables.

Syntax:
a, b, c = (1, 2, 3)

3. What happens if variables don't match tuple elements?

Answer: Python raises a **ValueError** because the number of variables and values must be equal.

4. How does * (extended unpacking) work?

Answer: It collects multiple values into a list.

Example:

a, *b = (1, 2, 3, 4)
a = 1, b = [2, 3, 4]

5. Can tuple packing be done without parentheses?

Answer: Yes, parentheses are optional.

Example:

t = 1, 2, 3

6. Difference between packing and unpacking?

Answer:
- Packing → combining values into a tuple
- Unpacking → extracting values into variables

7. Swap two variables using unpacking.

Answer:
```
a = 5
b = 10
a, b = b, a
```

8. What is nested tuple unpacking?

Answer: Unpacking tuples inside tuples.

Example:

```
t = (1, (2, 3))
a, (b, c) = t
```

9. Tuple unpacking in function return?

Answer: Functions can return multiple values as a tuple.

Example:

```
def fun():
    return 1, 2
a, b = fun()
```

10. Role of underscore _?

Answer: It is used to ignore unwanted values.

Example:

```
a, _, c = (1, 2, 3)
```

11. Can you unpack inside a loop?

Answer: Yes, especially with lists of tuples.
Example:

```
data = [(1, 2), (3, 4)]
for a, b in data:
    print(a, b)
```

12. What error occurs if unpacking fails?

Answer: ValueError occurs when values and variables mismatch.

13. How to ignore specific values?

Answer: Use _ or *.

Example:

```
a, _, c = (1, 2, 3)
```

14. Can tuples have different data types?

Answer: Yes, tuples can store mixed data types.

Example:

```
t = (1, "hello", 3.5)
a, b, c = t
```

15. Real-world use of packing & unpacking?

Answer:

- Returning multiple values from functions
- Swapping variables
- Iterating structured data
- Cleaner, shorter code